DNN Partitioning for GPU–CPU Heterogeneous Devices Based on Imitation Learning

Liying Li, Tao Liu, Mingzhou Zhao, Peijin Cong, Zebin Wu, *Senior Member, IEEE*, and Junlong Zhou, *Member, IEEE*

***Abstract*—The widespread adoption of deep neural networks (DNNs) across diverse applications has intensified the demand for efficient execution strategies on graphics processing unit (GPU)–CPU heterogeneous devices. While existing device-oriented DNN partitioning methods optimize either energy consumption or execution latency in isolation, they often lack the capability to dynamically adapt partitioning strategies under real-world constraints. To address these limitations, we first formulate the DNN partitioning problem as an optimal execution path search on a directed acyclic graph (DAG), and then propose an imitation learning (IL)-based DNN partitioning framework that dynamically allocates DNN layers across GPU–CPU processors to minimize energy consumption while satisfying time constraints. Specifically, we first design a *Profiler*, which analyzes the layerwise characteristics such as the execution time and energy consumption of layers in DNNs executed on the GPU–CPU heterogeneous device. Then, we present an *Oracle Generator* that generates high-quality partitioning solutions based on the Dijkstra algorithm to form a training dataset *Oracle*. *Oracle* from the proposed generator is input to a proposed *Trainer*, which iteratively learns from the *Oracle* data to obtain a regression model named *Predictor*. *Predictor* can predict the optimal partitioning strategy for DNNs in real time. Finally, we design a *Data Aggregator*, which enhances *Oracle* data through continuous runtime feedback, thereby improving the prediction accuracy of the regression model. We evaluate the proposed IL-based DNN partitioning method on three NVIDIA Jetson platforms and one Huawei NPU platform. Experimental results demonstrate that compared with three benchmarking methods, our method reduces energy consumption by up to 73.15%.**

***Index Terms*—Deep neural network (DNN) partitioning, energy optimization, graphics processing unit (GPU)–CPU heterogeneous device, imitation learning (IL).**

I. Introduction

In THE era of rapid development of artificial intelligence, deep neural networks (DNNs) have achieved remarkable success in diverse applications such as traffic prediction, attack detection, and Internet of Visual Things [1], [2], [3]. The proliferation of edge computing and Internet of Things (IoT) has further accelerated the trend of deploying DNNs on devices [4]. However, the increasing computational demands of modern DNNs often exceed the resources of individual CPU processors, driving widespread adoption of graphics processing unit (GPU)–CPU heterogeneous devices. To provide more efficient computing systems, hardware vendors such as NVIDIA and Xilinx are rolling out more powerful devices. These devices (such as NVIDIA Jetson AGX Xavier) typically integrate heterogeneous CPUs and GPUs in a single system-on-chip. Nevertheless, the deployment of DNNs on these GPU–CPU heterogeneous devices still presents fundamental energy and latency challenges. On one hand, while GPUs demonstrate superior time and energy efficiency for parallelizable operations such as convolutions, their fixed power budgets and memory access patterns lead to suboptimal performance for irregular computations. This often results in either excessive energy consumption or undesirable latency when processing layers such as fully connected layers. Conversely, CPUs exhibit lower energy consumption for control-intensive tasks but cannot match GPU throughput for parallel workloads [5]. This architectural dichotomy creates an optimization dilemma: executing entire networks on either processor type inevitably sacrifices either energy efficiency or real-time responsiveness. On the other hand, the interconnect between processors introduces extra transfer overheads. We measure the execution time of the first 20 layers in U-Net on the CPU and GPU of NVIDIA Jetson AGX Xavier separately, along with the time for transmitting feature

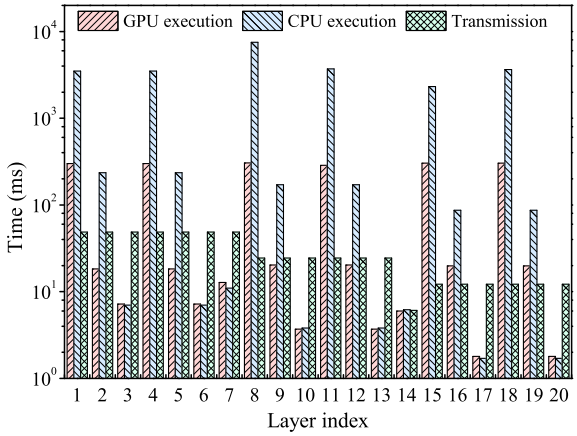


Fig. 1. GPU/CPU execution time and the transmission time of the first 20 layers in U-Net on NVIDIA Jetson AGX Xavier, where the transmission time means the time for transmitting each layer's feature map/output from the CPU to the GPU.

Received 28 August 2025; accepted 16 October 2025. Date of publication 22 October 2025; date of current version 8 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U23B2006, Grant 62302216, Grant 62302221, and Grant 62172224; in part by the Natural Science Foundation of Jiangsu Province under Grant BK20230912, Grant BK20230913, and Grant BK20220138; in part by the Fundamental Research Funds for the Central Universities under Grant 30925010515 and Grant 30924010815; and in part by the Open Research Fund of National Mobile Communications Research Laboratory, Southeast University under Grant 2024D07. *(Corresponding authors: Peijin Cong; Junlong Zhou.)*

Liying Li, Tao Liu, Mingzhou Zhao, Peijin Cong, and Zebin Wu are with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: cpj@njust.edu.cn).

Junlong Zhou is with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China, and also with the National Mobile Communications Research Laboratory, Southeast University, Nanjing 211111, China (e-mail: jlzhou@njust.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3624053

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

maps/outputs from the CPU to the GPU, as shown in Fig. 1. It can be seen from the figure that the transmission latency for some layers is even higher than that of the processing latency.

DNN partitioning is a promising approach to address the above challenges. To date, research efforts have pursued two primary directions for DNN partitioning. On one hand, cloud–edge collaborative methods [8], [9], [10], [12], [13], [14] offload DNN computations between edge devices and remote servers. However, these approaches, which are cloud–edge collaboration frameworks, falter in latency-sensitive or bandwidth-constrained scenarios due to the long transmission time. On the other hand, DNN partitioning techniques on devices optimize local resource usage by splitting DNNs into segments executed on different processors [5], [15], [16], [17]. However, existing device-oriented methods optimize either energy consumption or execution latency in isolation. In addition, they often lack the capability to dynamically adapt DNN partitioning strategies under real-world constraints.

To address these challenges, we turn to imitation learning (IL) [6]. Without loss of generality, IL can be broadly categorized into three types: behavioral cloning, inverse reinforcement learning (RL), and interactive IL [7]. In this article, we use interactive IL, which iteratively observes and interacts with an expert for learning. Interactive IL is a paradigm uniquely suited to this problem for three reasons. First, the combinatorial complexity of DNN partitioning—where each layer's placement affects overall performance—renders traditional optimization techniques such as RL computationally prohibitive for real-time use. Interactive IL learns from precomputed high-quality expert solutions (i.e., Oracle data), enabling rapid online inference. Second, interactive IL 's ability to generalize from training data allows our framework to adapt to diverse DNN architectures without retraining, simply by correlating layer features (e.g., input/output sizes, operator types) with optimal partition points. Third, unlike static approaches, our IL-based system continuously refines its predictions through runtime feedback (e.g., latency constraint violations), ensuring robustness in dynamic edge environments. Specifically, the proposed method enables feedback using a designed data aggregator, which provides additional guidance during training, allowing the proposed method to refine the partitioning policy incrementally. Together, these properties make interactive IL an ideal foundation for adaptive DNN partitioning.

In this article, we design a DNN partitioning scheme for GPU–CPU heterogeneous devices based on IL. The proposed method dynamically allocates DNN layers across GPU–CPU processors to minimize energy consumption while satisfying time constraints. The main contributions of this article are summarized as follows.

1) We design an IL-based DNN partitioning scheme, which can convert optimal DNN partitioning solutions (i.e., Oracle data) into DNN execution paths on a GPU–CPU heterogeneous device to address the energy optimization problem at the premise of satisfying the time constraints.
2) We design a *Profiler*, which analyzes the layerwise characteristics of DNN layers executed on the GPU–CPU heterogeneous device. We present an *Oracle Generator* that generates high-quality partitioning solutions based on the Dijkstra algorithm to form a training dataset named Oracle. Oracle is input to a proposed *Trainer* to obtain a regression model named *Predictor*, which predicts the optimal partitioning strategy for DNNs in real time. We design a *Data Aggregator*, which enhances Oracle data through continuous runtime feedback, thereby improving the prediction accuracy of the *Predictor*.
3) We conduct extensive experiments on three NVIDIA Jetson series platforms (i.e., AGX Orin, Xavier, and NX) for performance validation and one Huawei NPU platform (i.e., Atlas 200I DK A2) for method scalability discussion. The experimental results demonstrate that our proposed IL-based DNN partitioning method reduces energy consumption by up to 73.15%, compared with three benchmarking methods. We also conduct a case study to demonstrate the specific partition process and verify the time constraints.

The rest of this article is organized as follows. Section II reviews the work in related fields in recent years. Section III introduces the system model of this article. Section IV defines the problem and introduces the framework of the proposed algorithm. Section V presents the details of the proposed IL-based DNN partitioning method. Section VI validates the proposed IL-based DNN partition scheme through extensive experiments. Section VII summarizes this article.

II. Related Work

Extensive efforts have been made to promote the application of DNNs by designing DNN partitioning algorithms. Kakolyris et al. [8] introduced a method for DNN partitioning and offloading. This method uses prior knowledge and collaborative filtering to fastly predict energy demands of each layer in DNN across heterogeneous devices. It identifies a suite of Pareto-optimal DNN partitioning strategies that strike a balance between accuracy and energy consumption. Mao et al. [9] proposed a distributed mobile computing framework for DNNs. They divided a trained DNN into multiple parts and deployed these parts into different mobile devices for acceleration. They also designed DNN partition schemes for reducing the transfer time of nonparallel data. Within the cloud–edge–device architecture, Kang et al. [10] designed a scheduler, which can partition the DNN into two parts. The partitioned two parts are deployed on a device and cloud servers, respectively. Eshratifar et al. [12] proposed an adaptive engine, which is used to divide the collaborative calculation of the DNN's inference and the training phases between devices and the cloud. They also provided granular layer optimization formulations for two propagation processes in DNNs, which can adapt to energy limitation of devices, load balance of the cloud, and service quality. A distributed computing framework based on the cloud, edge, and devices is designed in [13]. This method maps various parts of the DNN to the distributed computing hierarchy and minimizes device communication and resource usage through joint training. Zhang et al. [14] proposed an edge–cloud collaborative intelligence method, which compresses and splits the convolutional neural network

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

(CNN) model for low latency in IoT. This method first speeds the deep model by generating new layers using low-rank decomposition techniques, and then establishes a fine-grained prediction model to find the optimal partition. This method optimally splits the compressed CNN model into two submodels. The obtained submodels are deployed on a device and the cloud server and work together to reduce the delay without degrading the model accuracy. The above methods have achieved relatively efficient intelligent network layer partitioning under the cloud–edge–device architecture. However, these works are designed for device–cloud architecture and not suitable for heterogeneous GPU–CPU edge devices.

Actually, with the rapid development of edge computing, more and more applications require deploying DNNs on GPU–CPU heterogeneous devices to achieve better performance. Some research efforts have been dedicated to the partitioning of DNNs on GPU–CPU heterogeneous devices. Wang et al. [5] conducted numerous experiments to study the system power consumption of layers in YoloV3-Tiny. Their observations substantiate the applicability of network partitioning techniques to heterogeneous edge devices and offer guidance for further design. Xiang and Kim [15] introduced a pipeline-based scheduling approach for GPU–CPU heterogeneous devices. They partitioned DNNs into multiple segments and scheduled them on the nodes to achieve real-time inference. The scheduling architecture also encompasses a set of features, including pipeline design, device resource configuration, execution time profiling, and runtime optimization mechanisms. Zhang et al. [16] proposed a DNN inference acceleration approach for heterogeneous GPU–CPU devices. This method includes a computation latency prediction scheme and a task scheduling approach. Yu et al. [17] proposed an asymmetric multiprocessing framework, which accommodates GPU and CPU in hierarchical computing and memory aspects for parallel large language model Inference at the user end. Their designed strategy enhances CPU and GPU utilization and fully hides the data loading time behind the CPU compute time. The above partitioning methods improve the performance of DNN tasks on heterogeneous GPU–CPU devices to some extent. However, the above works ignore the critical aspects of latency and/or energy consumption, which are actually very important for DNN applications on devices.

A few studies have been devoted to solving these problems. In response to the latency of devices, Jiang et al. [18] proposed a distributed DNN training framework, which uses computation and communication resources for DNN training acceleration. Furthermore, Kaur and Mohammadi [19] presented an in-depth analysis of energy efficiency and performance of a heterogeneous GPU–CPU platform with different workload distribution settings to determine optimal configuration. Wu et al. [20] proposed a DNN adaptive GPU–CPU optimization technique to find the Pareto-optimal speed and energy consumption in inference process of DNN. These methods consider the latency and/or energy consumption of DNN tasks on GPU–CPU heterogeneous devices. However, the above DNN partitioning methods are incapable of adjusting their generated DNN partitioning strategies under real-world constraints.

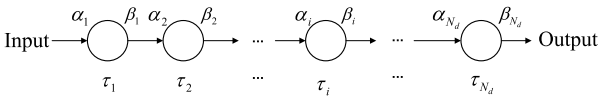


Fig. 2. Toy example of the linear topology DNN task.

III. System Models

In this section, the system models adopted in this work, including the device model, DNN task model, time model, and energy model, are introduced.

A. Device and DNN Task Models

As for computing platform, we consider a GPU–CPU heterogeneous device that integrates one GPU and M homogeneous CPUs. The GPU is represented by ρ_0, which supports V frequencies $\{f_1^G, f_2^G, \ldots, f_V^G\}$ $(f_1^G \le f_2^G \le \cdots \le f_V^G)$. M CPUs $\{\rho_1, \rho_2, \ldots, \rho_M\}$ are homogeneous. The ρ_m $(1 \le m \le M)$ supports W frequencies $\{f_1^C, f_2^C, \ldots, f_W^C\}$, where $f_1^C \le f_2^C \le \cdots \le f_W^C$ holds. CPUs provide general computing power, and the GPU is responsible for handling parallel computing tasks. The CPUs and GPUs are assumed to support dynamic voltage/frequency scaling (DVFS) technology, which adjusts one processor's voltage and frequency based on its workload. Typical examples of platforms integrating CPU and GPU cores are NVIDIA's Jetson series, such as Nano, Tegra, NX, and AGX [22].

As for tasks, we mainly consider DNN tasks. Assume a DNN set consists of D DNN tasks $\mathbb{D} = \{\Gamma_1, \Gamma_2, \ldots, \Gamma_D\}$. Each DNN task consists of a series of layers with a linear topology. A toy example of the linear topology DNN task is given in Fig. 2. DNN Γ_d contains N_d layers, and the ith layer in the network is represented by τ_i $(1 \le i \le N_d)$. Layers in the DNN task are performed in order. The input and output sizes of layer τ_i are α_i and β_i, respectively. Obviously, a layer's output is the next layer's input. Note that the input of the first layer is the input of the DNN task, and the output of the last layer is the output of the DNN task. We define $\mathbf{Z}$ as an $N_d \times 3N_d$ 0–1 matrix to describe the dependencies of layers in the DNN. The ith row and jth column element in $\mathbf{Z}$, denoted by Z_{ij} $(1 \le i, j \le N_d)$, is 1 when the ith layer (τ_i) has dependency with the jth layer (τ_j), and τ_i will be executed before τ_j. $Z_{ij} = 0$ indicates τ_i has no dependency with τ_j. Note that although the DNN task model in this article is simplified as a linear topology, DNNs with skip connections (e.g., ResNet) are treated as a whole layer. In this way, we can convert these nonlinear networks into linear networks for processing.

Each layer in a DNN task is executed on a specific processor. Our goal is to find a multilayer partition method for each DNN task on the GPU–CPU heterogeneous device to optimize the task's energy consumption. In other words, we need to find the optimal partition point for each DNN task and determine on which processor each layer in the DNN is

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

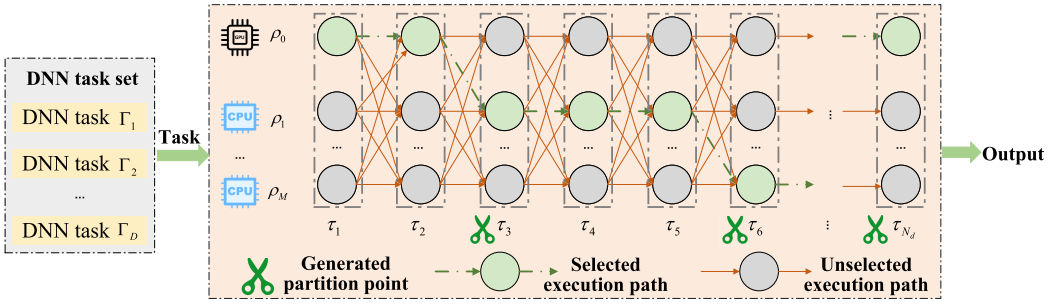


Fig. 3. Modeling the execution path of a DNN task on a GPU–CPU heterogeneous device as a DAG diagram.

executed. To this end, we denote the execution of each DNN task Γ_d ($1 \le d \le D$) on the GPU–CPU heterogeneous device as a directed acyclic graph (DAG) $G(V_d, \mathbf{Y}_d)$. V_d is the set of nodes in the DAG, which is in fact the DNN task Γ_d on the heterogeneous device of $M+1$ processors. V_d contains $(M+1) \times N_d$ nodes, where N_d is the number of layers in Γ_d. v_{mn} ($0 \le m \le M$, $1 \le n \le N_d$) denotes the node of the nth layer (i.e., τ_n) in Γ_d executed on the mth processor (i.e., ρ_m). $\mathbf{Y}_d$ is the set of edges among layers in DNN task Γ_d. $Y_{ij} \in \mathbf{Y}_d$ ($1 \le i, j \le N_d$) represents the transmission latency between τ_i and τ_j. Y_{ij} can be derived by

$$Y_{ij} = \beta_i / v_{ij} \tag{1}$$

where β_i is the output size of layer τ_i. The v_{ij} is the transmission speed between processor executes τ_i and processor executes τ_j. If τ_i and τ_j are executed on one processor, Y_{ij} is 0.

The execution path of DNN task Γ_d on the GPU–CPU heterogeneous platform can be illustrated in Fig. 3. The DNN task Γ_d of N_d layers from the task set is input into the GPU–CPU heterogeneous device. The gray nodes and the corresponding edges represent the other possible but not selected execution paths of Γ_d on the device. The green nodes and the corresponding edges consist of the Γ_d's execution path on the device. Let x_i ($1 \le i \le N_d$) be an execution location indicator, where $x_i = 0$ means layer τ_i is executed on the GPU ρ_0, while $x_i = m$ ($1 \le m \le M$) indicates τ_i is executed on CPU ρ_m. The partition point means that at this layer, the DNN task needs to change a core for execution. Note that we partition DNN tasks by layers, and the partition point is the layer whose execution changed. For example, as shown in Fig. 3, the first partition point of Γ_d is τ_3, which means layers before the partition point (i.e., τ_1 and τ_2) are executed on one core (i.e., ρ_0), and τ_3 and the layers after τ_3 are executed on other cores on the device. In this manner, the output of τ_2 needs to be transmitted from ρ_0 to CPU ρ_1.

B. Time Model

We mainly consider the time of performing DNN tasks on the GPU–CPU heterogeneous device, which consists of the time executing layers on processors and the time transferring data between processors. The execution time of executing layer τ_i ($1 \le i \le N_d$) in DNN task Γ_d is

$$e_i = \frac{c_i}{f_\rho} \tag{2}$$

where c_i is the number of clock cycles needed for executing τ_i. f_ρ denotes the frequency of the processor executing τ_i. The transmission time is calculated using (1). The total time of performing DNN task Γ_d on the GPU–CPU heterogeneous device can be calculated by

$$T_{\Gamma_d}^{\mathrm{total}} = \sum_{i=1}^{N_d} e_i + \sum_{i=1}^{N_d} \sum_{j=1}^{N_d} Y_{ij}. \tag{3}$$

C. Energy Model

We mainly consider the energy consumption of executing DNNs on the GPU–CPU heterogeneous device. The total power consumption of the device is composed of CPU's and GPU's power consumption, i.e.

$$P_{\mathrm{tol}}(t) = P_G(t) + \sum_{m=1}^{M} P_m(t) \tag{4}$$

where $P_G(t)$ is the power consumption of the GPU ρ_0 at time t, and $P_m(t)$ denotes CPU ρ_m's power consumption at time t. The power of one processor mainly includes dynamic and leakage power [5]. Dynamic power denotes the power consumed for executing tasks on the processor. And the leakage power means the power consumed when the processor is idle. Similar to the power consumption model in [5] and [23], the power consumption of processor ρ is derived by

$$P_\rho(t) = P_\rho^{\mathrm{dyn}}(t) + P_\rho^{\mathrm{leak}}(t) \tag{5}$$

where $P_\rho^{\mathrm{dyn}}(t)$ and $P_\rho^{\mathrm{leak}}(t)$ are the dynamic power and leakage power of processor ρ at time t, respectively. $P_\rho^{\mathrm{dyn}}(t)$ can be derived by

$$P_\rho^{\mathrm{dyn}}(t) = u_\rho \times \delta_\rho \times V_\rho(t) \times f_\rho(t) \tag{6}$$

where u_ρ denotes the utilization of processor ρ. δ_ρ is a processor-related constant. $V_\rho(t)$ and $f_\rho(t)$ represent the voltage and frequency of processor ρ at time t, respectively. Leakage power consumption is caused by the leakage current inside the processor, which is related to the temperature of the processor. $P_\rho^{\mathrm{leak}}(t)$ can be calculated by

$$P_\rho^{\mathrm{leak}}(t) = V_\rho(t) \times \left(\sigma_\rho + \gamma_\rho \times T_\rho(t)\right) \tag{7}$$

where σ_ρ and γ_ρ are the processor-related constants. $T_\rho(t)$ denotes the temperature of processor ρ at time t.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

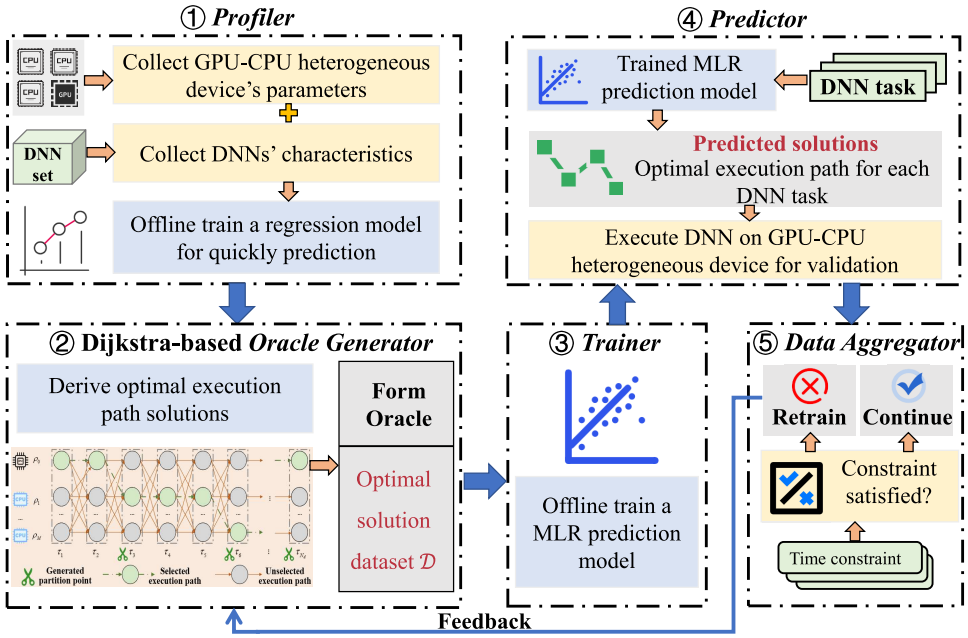


Fig. 4. Overview of the proposed IL-based DNN partitioning method.

The total energy consumption of the device, denoted by E_{tol}, can be derived by

$$E_{\text{tol}} = \int_{t_0}^{t_N} P_{\text{tol}}(t)\, \mathrm{dt} \tag{8}$$

where t_0 and t_N are the start time and the end time of the task execution, respectively. $P_{\text{tol}}(t)$ is derived by (4).

IV. Problem Definition and Algorithm Framework

In this section, the problem to be tackled in this article is first formalized. Then we give a framework of the proposed method based on IL.

A. Problem Definition

We aim at minimizing the energy consumption of each deep neural network executed on GPU–CPU heterogeneous devices while meeting the time requirements. We mathematically formulate the problem to be solved in this article as

$$\text{Minimize: } E_{\text{tol}} \tag{9}$$

$$\text{subject to: } T_{\Gamma_d}^{\text{total}} \le T_{\text{th}} \quad \forall\ \Gamma_d \in \mathbb{D} \tag{10}$$

$$e_i \le T_{\text{th}}^i \quad \forall\ 1 \le i \le N_d \tag{11}$$

where E_{tol} is the total energy consumption of a DNN task from the DNN task set. E_{tol} can be calculated by (8). $T_{\Gamma_d}^{\text{total}}$ denotes the total time of performing DNN task Γ_d on the GPU–CPU heterogeneous device, which can be derived by (3). T_{th} represents a predefined threshold. Therefore, (10) means that the total time of performing each DNN task Γ_d on the heterogeneous device should not exceed T_{th}. e_i is the execution time of layer τ_i in DNN task Γ_d, which can be derived by (2). T_{th}^i is the predefined threshold of the execution time of layer τ_i in DNN task Γ_d. Therefore, (11) means that the execution time of each layer τ_i in DNN task Γ_d should not exceed a predefined threshold T_{th}^i.

B. Algorithm Framework

Fig. 4 shows the framework of the proposed method based on IL. The proposed method mainly consists of five parts: *Profiler*, *Oracle Generator*, *Trainer*, *Predictor*, and *Data Aggregator*. *Profiler* is responsible for collecting and analyzing characteristics such as the execution time and energy consumption of each layer in each DNN task executed on the GPU–CPU heterogeneous device. These characteristics are input to the *Oracle Generator*. *Oracle Generator* is designed to generate the optimal partition point and derive an optimal execution path solution for each DNN task. The optimal execution paths of all the DNN tasks form the partitioning solution dataset, named Oracle. Most traditional methods based on IL use exhaustive enumeration as their Oracle generator method. However, exhaustive enumeration is very time-consuming. In this article, we propose a Dijkstra-based method to generate the Oracle dataset quickly. The obtained Oracle dataset is then input to *Trainer* to train a regression model, which is based on multifeature linear regression (MLR). *Predictor* is responsible for predicting optimal partition points and execution paths for DNN tasks using the trained prediction model. *Data Aggregator* is responsible for checking whether the DNN execution path obtained by the predictor can meet the time constraints when executing the DNN on the GPU–CPU heterogeneous device. If the time constraints are met, the DNN execution path is output as the final solution. Otherwise, *Data Aggregator* feedbacks the solution to *Oracle Generator* to dynamically supplement training data. *Trainer* dynamically updates the regression model based on the supplemented training data for better performance.

V. IL-Based DNN Partitioning

This section introduces our proposed IL-based DNN partitioning scheme in detail.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

A. Profiler and Oracle Generator

Profiler acts like a characteristic analyzer, which collects and analyzes the characteristics such as the execution time and energy consumption of each layer in each DNN task executed on the GPU–CPU heterogeneous device. The simplest way to obtain the characteristics is to execute each DNN task on the GPU–CPU heterogeneous device and collect the characteristics. However, executing all the DNN tasks is time-consuming and energy-consuming. The designed *Profiler* trains a regression model to quickly predict the characteristics according to the parameters of DNN tasks and devices. Specifically, the model first collects the parameters and the input and output sizes of each layer in each DNN task. The parameters of GPU–CPU heterogeneous devices are also collected. These characteristics are used as features. Each DNN task is then executed on the GPU and CPU, respectively, and the actual energy consumption and execution time of each layer on the GPU and CPU are collected, which are used as labels. The collected features and labels are input to the model for offline training. Note that in our design and intended application scenarios, the offline training phase of *Profiler* is envisioned to be conducted once per model–hardware pair on a powerful and energy-unconstrained platform, such as a cloud server or a dedicated development machine. The resulting pretrained profiling model is then deployed to devices for online inference. The trained *Profiler* model can quickly predict the energy consumption and execution time of each layer in DNN tasks executing on the GPU and CPU online. The predicted results are input to the proposed *Oracle Generator*. Note that the performance of the proposed *Profiler* is important to the overall performance of the proposed IL-based DNN partitioning method. If the prediction error of *Profiler* is large, the obtained DNN partitioning solution may become suboptimal. Therefore, we evaluate the performance of the proposed *Profiler* in Section VI-B.

As aforementioned, *Oracle Generator* is designed to generate the expert policy (i.e., optimal partition point) and derive an optimal execution path solution for each DNN task using the Dijkstra-based method. It generates an optimal solution dataset named Oracle. Specifically, the proposed Dijkstra-based method takes DNN task set $\mathbb{D} = \{\Gamma_1, \Gamma_2, \ldots, \Gamma_D\}$, time constraints T_{th} and T_{th}^i, and corresponding DAGs, denoted by $\{G(V_1, \mathbf{Y}_1), G(V_2, \mathbf{Y}_2), \ldots, G(V_D, \mathbf{Y}_D)\}$, as inputs, and outputs the optimal solution dataset $\mathcal{D}$. The pseudocode of the Dijkstra-based Oracle generator is shown in Algorithm 1. For each DNN task Γ_d ($1 \le d \le D$), the algorithm first initializes the dataset $\mathcal{D}$ as empty and sets the counter $n = 1$ in line 2. Then we use the proposed *Profiler* to predict the energy consumption and execution time (e_1) of the first layer of Γ_d on $M+1$ processors (line 3). The processor with the minimum energy consumption is selected, which is denoted by $p_{d,1}$ (line 4). The distance value for Γ_d is then initialized as e_1, and the starting point $p_{d,1}$ is added to the dataset $\mathcal{D}$ (lines 5 and 6). After determining the starting point, the algorithm iterates over the remaining layers of Γ_d from line 7 to line 14. Specifically, for each layer τ_n ($2 \le n \le N_d$), the algorithm uses the *Profiler* to predict the energy consumption and execution time of τ_n on $M+1$ processors (line 8). Algorithm 1 then selects the processor ($p_{d,n}$) with the minimum energy consumption and adds $p_{d,n}$ to $\mathcal{D}$ (lines 9 and 10). Afterward, the algorithm uses the *Profiler* to predict the execution time of τ_n on processor $p_{d,n}$ (line 11) and calculates the transmission time between τ_{n-1} and τ_n on $p_{d,n-1}$ and $p_{d,n}$ by (1) (line 12). The total time T^{total} is calculated by (3) (line 13), and the distance value Dist_d is updated by adding T^{total} (line 14). Finally, the optimal solution dataset $\mathcal{D}$ is returned (line 15).

Algorithm 1 Dijkstra-Based Oracle Generator

Input: DNN task set $\mathbb{D} = \{\Gamma_1, \Gamma_2, ..., \Gamma_D\}$, time constraints, corresponding DAGs $\{G(V_1, \mathbf{Y}_1), G(V_2, \mathbf{Y}_2), ..., G(V_D, \mathbf{Y}_D)\}$;
Output: Optimal solution dataset $\mathcal{D}$;

1 **for** $d = 1$; $d \le D$; $d++$ **do**
  // Initialization.
2 $\mathcal{D} = \emptyset$, $n = 1$;
3 Utilize the proposed *Profiler* to predict the energy consumption and execution time (e_1) of τ_1 on $M+1$ processors;
4 Select the processor with the minimum energy consumption and set the processor as $p_{d,1}$;
5 $Dist_d = e_1$;
6 Add the starting point $p_{d,1}$ to the $\mathcal{D}$;
7 **for** $n = 2$; $n \le N_d$; $n++$ **do**
8 Utilize the proposed *Profiler* to predict the energy consumption and execution time of τ_n on $M+1$ processors;
9 Select the processor with the minimum energy consumption and set the processor as $p_{d,n}$;
10 Add $p_{d,n}$ to $\mathcal{D}$;
11 Utilize the proposed *Profiler* to predict the execution time of τ_n on processor $p_{d,n}$;
12 Calculate the transmission time between τ_{n-1} and τ_n on $p_{d,n-1}$ and $p_{d,n}$ by Eq. (1);
13 Calculate the total time T^{total} by Eq. (3);
14 $Dist_d = Dist_d + T^{total}$;
15 **return** *Optimal solution dataset* $\mathcal{D}$;

The Dijkstra-based Oracle generator generates the optimal execution path solution for each DNN task. Let $\mathbf{p}_d = \{p_{d,1}, p_{d,2}, \ldots, p_{d,N_d}\}$ and $\mathbf{F_d} = [F_d^1, F_d^2, \ldots, F_d^{N_F}]$ be the optimal execution path solution and features for DNN task Γ_d, respectively. N_F is the number of features. The feature $\mathbf{F}_d$ contains the parameters of the GPU–CPU heterogeneous device, the characteristics of each layer in Γ_d, and the input and output sizes of each layer in Γ_d. The optimal solution dataset $\mathcal{D}$ can be represented as $\mathcal{D} = \{(\mathbf{F}_1, \mathbf{p}_1), (\mathbf{F}_2, \mathbf{p}_2), \ldots, (\mathbf{F}_D, \mathbf{p}_D)\}$. The optimal execution path solutions for all the DNN tasks form the Oracle dataset, which is input to the *Trainer* for training a regression model.

B. Trainer and Predictor

As aforementioned, the proposed Dijkstra-based *Oracle Generator* generates an optimal solution dataset $\mathcal{D}$. The solution dataset $\mathcal{D}$ generated by the proposed *Oracle Generator* is not suitable to be stored in devices due to the resource limitation. Therefore, the optimal solution dataset $\mathcal{D}$ is input to the *Trainer* for training a regression model. The training process of the proposed *Trainer* is shown in Fig. 5. Specifically,

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

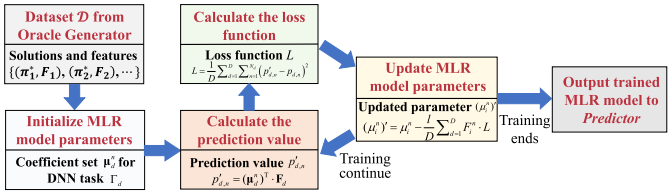


Fig. 5. Training process of the proposed *Trainer*.

we use MLR, which is a statistical technique that models the relationship between multiple features and a response variable, to train the regression model. The proposed *Trainer* iteratively learns from the dataset generated by the *Oracle Generator*. Let $\boldsymbol{\mu}_d^n = [\mu_1^n, \mu_2^n, \ldots, \mu_{N_F}^n]$ be the set of coefficients for the nth $(1 \le n \le N_d)$ element in $\mathbf{p}_d$. Then the prediction for $p_{d,n}$ can be derived by

$$p'_{d,n} = \left(\boldsymbol{\mu}_d^n\right)^{\mathrm{T}} \cdot \mathbf{F}_d \tag{12}$$

where $\mathbf{F}_d$ is the feature vector of Γ_d. $(\boldsymbol{\mu}_d^n)^{\mathrm{T}}$ denotes the transposition of $\boldsymbol{\mu}_d^n$. Note that the coefficient set $\boldsymbol{\mu}_d^n$ is randomly initialized. We define the loss function according to the prediction $p'_{d,n}$ and the actual value $p_{d,n}$ as

$$L = \frac{1}{D} \sum\nolimits_{d=1}^{D} \sum\nolimits_{n=1}^{N_d} \left(p'_{d,n} - p_{d,n}\right)^2 \tag{13}$$

where D is the number of DNN tasks in the dataset. The ith $(1 \le i \le N_F)$ coefficient in $\boldsymbol{\mu}_d^n$ is updated according to the gradient descent strategy [21] by

$$\left(\mu_i^n\right)' = \mu_i^n - \frac{l}{D} \sum\nolimits_{d=1}^{D} F_i^n \cdot L \tag{14}$$

where l is the learning rate. F_i^n $(1 \le d \le N_d)$ denotes the ith element in $\mathbf{F_d}$. After the iterations for the training process (i.e., all the training data in the optimal solution dataset $\mathcal{D}$ are processed), all the coefficients are fixed. The trained regression model is then input to the *Predictor* for predicting the optimal partition point and execution path solution for DNN tasks. Specifically, the *Predictor* predicts the nth optimal execution path solution for DNN task Γ_d by

$$p_{d,n} = \mu_1^n F_d^1 + \mu_2^n F_d^2 + \cdots + \mu_{N_F}^n F_d^{N_F} \tag{15}$$

where μ_i^n $(1 \le i \le N_F)$ are the fixed coefficients obtained by the regression model in *Trainer*.

The pseudocode of the proposed IL-based DNN partitioning algorithm is shown in Algorithm 2. It takes the DNN task set $\mathbb{D} = \{\Gamma_1, \Gamma_2, \ldots, \Gamma_D\}$, time constraints, parameters of the GPU–CPU heterogeneous device, and the DNN tasks to be partitioned as inputs, and outputs the optimal execution path for each DNN task. It first calls Algorithm 1 to generate the optimal solution dataset $\mathcal{D}$ (line 1). Then Algorithm 2 iteratively trains the regression model by using the training data in $\mathcal{D}$ (lines 2–6). The regression model iteratively learns from samples in the Oracle dataset $\mathcal{D}$. Specifically, for each DNN task Γ_d $(1 \le d \le D)$, the algorithm randomly initializes the coefficient set $\boldsymbol{\mu}_d^n$ for each layer in Γ_d (line 4). The loss function value L is calculated by (13) (line 5). The coefficient set $\boldsymbol{\mu}_d^n$ is updated by (14) (line 6). After the training process, Algorithm 2 predicts the optimal partition point and execution path solution for each DNN task using the trained regression model (lines 7–14). Specifically, for each DNN task to be partitioned, the algorithm predicts the optimal partition point and execution path solution for each layer in the DNN task using (15) in line 9. Afterward, the execution time of each layer and total time, denoted by e_i and T^{total}, respectively, of performing the DNN task according to the predicted optimal execution path solution is calculated (line 10). If $T^{\text{total}} \le T_{\text{th}}$ and all $e_i \le T_{\text{th}}^i$, the time constraints are met, and the predicted optimal execution path for the DNN task is returned (line 12). Otherwise, the algorithm calls Algorithm 3 (*Data Aggregator*) to dynamically supplement training data (line 14). The working mechanism of the proposed *Data Aggregator* will be introduced in Section V-C.

Algorithm 2 IL-Based DNN Partitioning Scheme

Input: DNN task set $\mathbb{D} = \{\Gamma_1, \Gamma_2, ..., \Gamma_D\}$, time constraints T_{th} and T_{th}^i, parameters of the GPU-CPU heterogeneous device, and the DNN tasks to be partitioned;
Output: Optimal execution path for each DNN task;
`// Profiler and Oracle Generator.`
1 Call Alg. 1 to generate the optimal solution dataset $\mathcal{D}$;
`// Trainer.`
2 **for** $d = 1$; $d \le D$; $d++$ **do**
3 **for** $n = 1$; $n \le N_d$; $n++$ **do**
4 Randomly initialize coefficient set μ_d^n for DNN task Γ_d;
5 Calculate the loss function value L by Eq. (13);
6 Update the coefficient set μ_d^n by Eq. (14);
`// Predictor.`
7 **for** each DNN tasks to be partitioned **do**
8 **for** each layer in the DNN task **do**
9 Predict the optimal partition point and execution path solution by Eq. (15);
`// Check the time constraints.`
10 Calculate the execution time (e_i) of each layer and total time (T^{total}) of performing the DNN task according to the predicted optimal execution path solution;
11 **if** $T^{\text{total}} \le T_{\text{th}}$ and all $e_i \le T_{\text{th}}^i$ **then**
12 **return** *The predicted optimal execution path for the DNN task*;
13 **else**
`// Data Aggregator.`
14 Call Alg. 3 and feedback the solution to the *Oracle Generator* to dynamically supplement training data;

Algorithm 3 Data Aggregator

Input: Feature $\mathbf{F}$, the predicted optimal execution path $\mathbf{p}$, the corresponding total time T^{total}, and the number of training samples in Oracle dataset $\mathcal{D}$;
Output: Updated Oracle dataset $\mathcal{D}$;
1 **while** time constraints are not met **do**
2 Refine the the predicted optimal execution path $\mathbf{p}$ by using Alg. 1;
3 Update the total time (T^{total}) of performing the DNN task according to the refined optimal execution path $\mathbf{p}$;
4 Add the refined optimal execution path $\mathbf{p}$ and the corresponding feature $\mathbf{F}$ to the Oracle dataset $\mathcal{D}$;
5 $D = D + 1$;
6 **return** *Updated Oracle dataset* $\mathcal{D}$;

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

C. Data Aggregator

The *Predictor* predicts the optimal partition point and execution path solution for DNN tasks. However, the predicted optimal execution path solution may not meet the time constraints when executing DNN on the GPU–CPU heterogeneous device. In this case, the *Predictor* feedbacks the solution to the *Oracle Generator* to dynamically supplement training data. *Data Aggregator* is responsible for refining the DNN execution paths that cannot meet the time constraints and adding the refined solutions to the Oracle dataset.

The pseudocode of the proposed *Data Aggregator* is shown in Algorithm 3. It takes the feature $\mathbf{F}$, the predicted optimal execution path $\mathbf{p}$, the corresponding total time T^{total}, and the number of training samples in Oracle dataset $\mathcal{D}$ as inputs, and outputs the updated Oracle dataset $\mathcal{D}$. The algorithm first refines the predicted optimal execution path $\mathbf{p}$ using Algorithm 1 (line 2). Then the total time (T^{total}) of performing the DNN task according to the refined optimal execution path $\mathbf{p}$ is updated (line 3). The algorithm iteratively refines the predicted optimal execution path $\mathbf{p}$ until the total time T^{total} meets the time constraints. After the refinement, the refined optimal execution path $\mathbf{p}$ and the corresponding feature $\mathbf{F}$ are added to the Oracle dataset $\mathcal{D}$ (line 4). The number of training samples in Oracle dataset $\mathcal{D}$ is updated by $D = D + 1$ (line 5). Finally, the updated Oracle dataset $\mathcal{D}$ is returned (line 6).

Note that in the scenarios we focus on, DNN partitioning is essentially an optimal path search problem based on a DAG, where the computation resource (GPU/CPU) allocation decision for each layer leads to an exponential combinatorial solution space (e.g., the theoretical number of partitioning schemes for an N_d-layer network is 2^{N_d}). Traditional DNN partitioning methods, such as RL-based methods, require extensive trial-and-error (e.g., state-action space exploration in Q-learning) to solve this problem, but their computational complexity grows exponentially with the number of layers, making it difficult to meet real-time inference requirements on resource-constrained edge devices. In contrast, our IL-based method leverages precomputed high-quality solutions (Oracle data, generated by the designed Dijkstra-based *Oracle Generator*) to transform the optimization problem into "learning a mapping from optimal examples," avoiding the trial-and-error overhead of RL. As shown in Fig. 4, the proposed method directly generates the optimal execution path through the *Profiler* and *Oracle Generator*, with a complexity of $O((M+1)^2)$ (where $M+1$ is the number of cores), which is significantly lower than the $O(2^{N_d})$ trial-and-error complexity of RL, especially advantageous for deep networks. Furthermore, RL relies on a stable environment model, and when dynamic factors such as device load or data transmission delay change, it requires retraining the policy network, making it difficult to adapt to real-time feedback needs on edge devices. In contrast, our IL-based DNN partitioning framework uses the *Data Aggregator* mechanism to dynamically supplement training data based on runtime feedback, allowing the *Predictor* model to quickly update partitioning strategies. In addition, IL-based methods generalize across different network architectures through layer features (input/output dimensions, operator types, etc.), while RL's policy network needs to be fine-tuned for specific network structures, resulting in weaker generalization capabilities.



Fig. 6. Adopted three NVIDIA GPU–CPU heterogeneous platforms and one Huawei NPU platform.

VI. Evaluation

In this section, we first introduce the experimental setups, including the experimental platforms, the comparison algorithms, and the evaluation metrics. Then we present the experimental results and analyze the performance of the proposed IL-based DNN partitioning method. Finally, we conduct a case study to demonstrate the partitioning process of the proposed method and verify the time constraints.

A. Evaluation Setups

To evaluate the performance of the proposed IL-based DNN partitioning method, we conduct extensive experiments on the NVIDIA Jetson series, including AGX Orin, AGX Xavier, and Xavier NX platforms [24]. Specifically, the AGX Orin development board adopts the ARM big.LITTLE architecture, which equips a 1792-core NVIDIA Ampere Architecture GPU with 56 Tensor Cores and a 12-core ARMv8.2 64-bit CPU. The AGX Xavier development board is equipped with 512-core Volta architecture GPUs and eight-core NVIDIA Carmel ARMv8.2 64-bit CPUs. The Xavier NX development board is equipped with 384 NVIDIA CUDA Cores, 48 Tensor Cores, six Carmel ARM CPUs, and two NVIDIA deep learning accelerator engines. To discuss the method's scalability, we also conduct the experiments on one Huawei NPU platform (Atlas 200I DK A2) [25]. The Atlas 200I DK A2 platform is equipped with an Ascend 310B1 NPU and four ARM Cortex-A55 CPUs. The data transfer speeds from one core to another on the experimental platforms are obtained through actual measurement. Fig. 6 shows the experimental platforms.

We select typical DNNs such as AlexNet and U-Net as DNN tasks for training, and DNNs such as VGG16 and DenseNet for testing. The main features of the selected DNN tasks are given in Table I. Note that we count the total number of layers in a DNN task using the `modules()` function in PyTorch, which

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

takes all the individual modules as separate layers. Although the DNN tasks in this article are simplified as a linear topology, DNNs with skip connections (e.g., ResNet) are treated as a whole layer. In this way, we can convert these nonlinear networks into linear networks for processing. In addition, to explore the performance of the proposed method on larger DNNs like transformer, we also select MobileNetVIT as a DNN task for testing. MobileNetVIT [11] is a general-purpose vision transformer for mobile devices, which is widely used in various computer vision tasks. To verify the effectiveness of the proposed IL-based DNN partitioning method, we compare it with three benchmarking methods. The three methods for comparison are introduced below.

1) *BASE:* This method is a baseline method, which executes DNNs on the GPU–CPU heterogeneous device without partitioning.
2) *Neurosurgeon [10]:* This method can automatically partition DNN into two parts according to an established prediction model and execute the two parts on the GPU–CPU heterogeneous device.
3) *D3PG [27]:* D3PG partitions DNN tasks based on a Dirichlet deep deterministic policy gradient algorithm. It formulates the DNN task partitioning and offloading problem as a constrained Markov decision process and uses a Dirichlet distribution to model the action space.

The data transmission speed between processors is determined through actual measurements. According to our measurements, we set the data transmission rate from CPU to CPU/GPU to 3.2 GB/s, and the data transmission rate from GPU to CPU is set to 3.4 GB/s. For different DNN tasks, since their characteristics such as architectures and parameter sizes are different, their expected execution time is different. Therefore, we set the time constraints for every DNN task according to its execution time before partitioned. Specifically, assume the total time of performing DNN task Γ_d on the GPU–CPU heterogeneous device is $T^{\text{total}}_{\Gamma_d}$, and the time constraint is set to $T_{\text{th}} = T^{\text{total}}_{\Gamma_d} \times (1 + \theta)$. The θ is a threshold ratio, which is set to 0.1, 0.2, and 0.3, respectively, in the following experiments. The time constraints for each layer are set similarly. Since the execution time of different layers on different cores is not fixed, we set the time constraints for each layer as multiples of the baseline execution time, which can better adapt to the execution time of different layers on different cores. To facilitate reproducibility and further research, the source code for the proposed method has been made open-source and is hosted on GitHub at: https://github.com/edgeflow-lab/imitation-dnn-partition

B. Verification of Profiler

As aforementioned, the proposed *Profiler* is used to predict the energy consumption and execution time of each layer in DNN tasks. The output of the *Profiler* is input to the *Oracle Generator* to generate the optimal solution dataset. Therefore, the prediction accuracy of the proposed *Profiler* affects the performance of the proposed IL-based DNN partitioning method. To verify the effectiveness of the proposed *Profiler*, we perform a series of experiments. Specifically, we compare the predicted energy consumption and execution time of each layer in DNN tasks with the actual values obtained by running the DNN tasks on the GPU–CPU heterogeneous device. Limited by the space, we only show energy prediction results of the convolutional layers and batch normalization layers on GPU and CPU, as shown in Fig. 7.

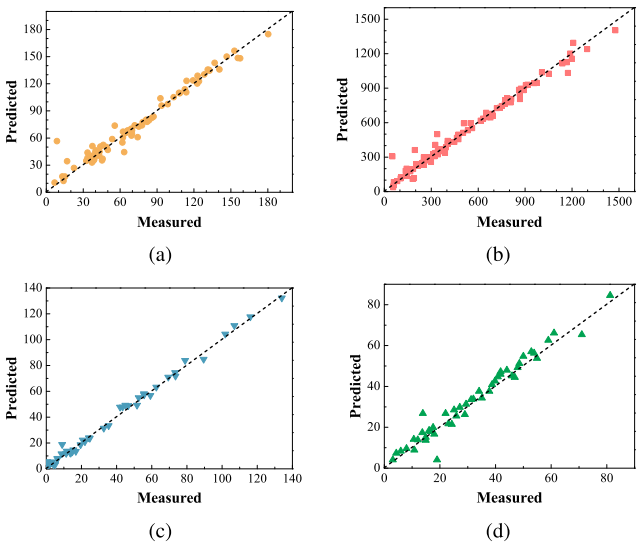


Fig. 7. Energy prediction performance of the proposed *Profiler*. (a) Energy prediction of convolutional layers on GPU. (b) Energy prediction of convolutional layers on CPU. (c) Energy prediction of batch normalization layers on GPU. (d) Energy prediction of batch normalization layers on CPU.

The X-axis is the measured value, while the Y-axis represents the values predicted by the proposed *Profiler*. The black dashed line in each figure represents the ideal prediction line. On this line, the predicted values are equal to the actual measured values. Consequently, the closer the points in the figure are to this line, the better the prediction performance. It can be seen from the figure that all the points are close to the ideal prediction line, which indicates that the proposed *Profiler* can accurately predict the energy consumption and execution time of each layer in DNN tasks. The prediction performance of the proposed *Profiler* is evaluated using the mean-squared error (mse) metric [26]. Note that the mse is calculated by the normalized predicted and actual values. These values are normalized using

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{16}$$

where x is the value to be normalized. The $x_{\min}$ and $x_{\max}$ are the minimum and maximum values, respectively. The trained energy prediction model of convolutional layers on GPU and CPU achieves mse of 7.92×10^{-4} and 2.48×10^{-4}, respectively. The trained energy prediction model of batch normalization layers on GPU and CPU achieves mse of 2.24×10^{-3} and 1.23×10^{-3}, respectively.

C. Experimental Results on Energy Consumption

To verify the effectiveness of the proposed IL-based DNN partition scheme, we compare the energy consumption of DNN tasks executed on the GPU–CPU heterogeneous device and

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

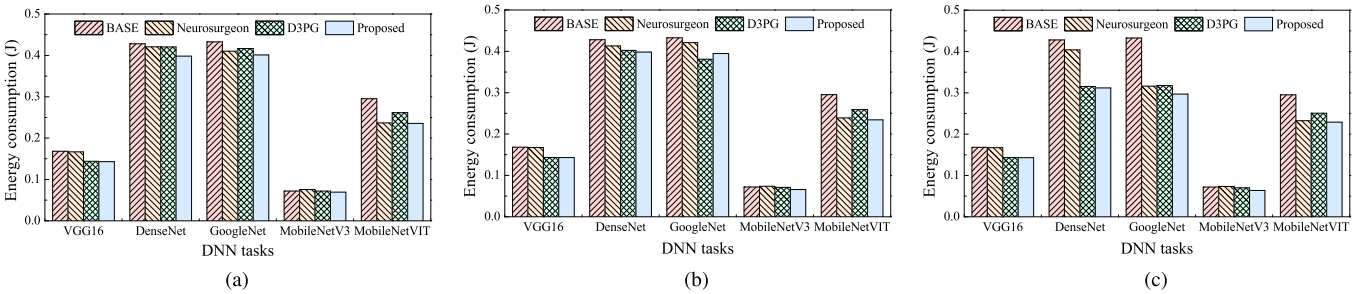


Fig. 8. Compare the energy consumption of executing different DNN tasks on AGX Orin using four methods under different time threshold ratios (θ). (a) θ = 0.1. (b) θ = 0.2. (c) θ = 0.3.

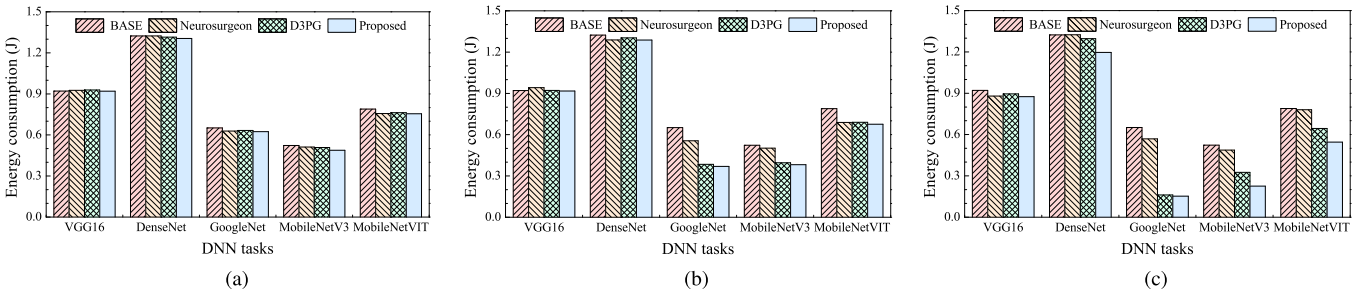


Fig. 9. Compare the energy consumption of executing different DNN tasks on AGX Xavier using four methods under different time threshold ratios (θ). (a) θ = 0.1. (b) θ = 0.2. (c) θ = 0.3.

on the Huawei NPU platform after using the proposed IL-based DNN partitioning method with that of the BASE and Neurosurgeon [10]. When the time threshold ratio (θ) is set to 0.1, 0.2, and 0.3, the energy consumption of executing different DNN tasks on AGX Orin by using the BASE, Neurosurgeon, D3PG, and the proposed methods is shown in Fig. 8. It can be seen from the figure that the proposed IL-based DNN partition method achieves lower energy consumption than the three benchmarking methods. When the time threshold ratio (θ) is set to 0.1, the proposed method can reduce energy consumption by up to 20.19%, 14.25%, and 9.95%, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. When θ is set to 0.2, the proposed method can reduce energy consumption by up to 20.63%, 14.53%, and 9.59%, compared with the three benchmarking methods, respectively. When θ is set to 0.3, the proposed method can reduce energy consumption by up to 31.37%, 22.76%, and 9.28%, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. Since the BASE method directly runs the unpartitioned DNN tasks on the GPU–CPU heterogeneous device, the energy consumption of this method does not change with the variation in the time threshold. Moreover, when compared with the other three methods, the BASE method consistently exhibits higher energy consumption across different DNN tasks. The Neurosurgeon method is a partitioning method that can only partition DNN tasks into two parts, which limits its ability to optimize energy consumption. The existence of time constraints may also limit the energy optimization performance of Neurosurgeon. That is to say, when a violation of the time constraint occurs, Neurosurgeon has to sacrifice its energy consumption optimization performance to meet the time constraint. For instance, from an energy consumption perspective, a certain layer of a DNN task should originally be run on the CPU. However, if this is done, it may violate the time constraints. The fact that Neurosurgeon only partitions the DNN task into two parts may exacerbate the negative impact of time constraints on energy consumption optimization. That is, Neurosurgeon is unable to dynamically adjust the partitioning strategy to achieve better energy consumption optimization results. The D3PG method is a partitioning method based on RL. However, the D3PG method cannot adaptively adjust the partitioning strategy according to the time constraints, which may lead to suboptimal energy consumption optimization results. The proposed IL-based DNN partitioning method can dynamically adjust the partitioning strategy according to the time constraints, which enables it to achieve better energy consumption optimization results. It uses the proposed *Data Aggregator* to dynamically supplement the training data, which allows the proposed method to adaptively adjust the partitioning strategy according to the time constraints. The time constraint verification will be analyzed in detail through two cases in Section VI-E.

When the time threshold ratio (θ) is set to 0.1, 0.2, and 0.3, the energy consumption of executing different DNN tasks on AGX Xavier is shown in Fig. 9. Similarly, the proposed IL-based DNN partitioning method achieves lower energy consumption than the BASE, Neurosurgeon, and D3PG methods. When θ is set to 0.1, the proposed method can reduce by up to 6.82%, 4.6%, and 3.94% energy consumption on average, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. Compared with the BASE, Neurosurgeon, and D3PG methods, the proposed method can reduce 17.57%, 12.43%, and 2.24% energy consumption on average, when θ is set to 0.2. When θ is set to 0.3, the proposed method can

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

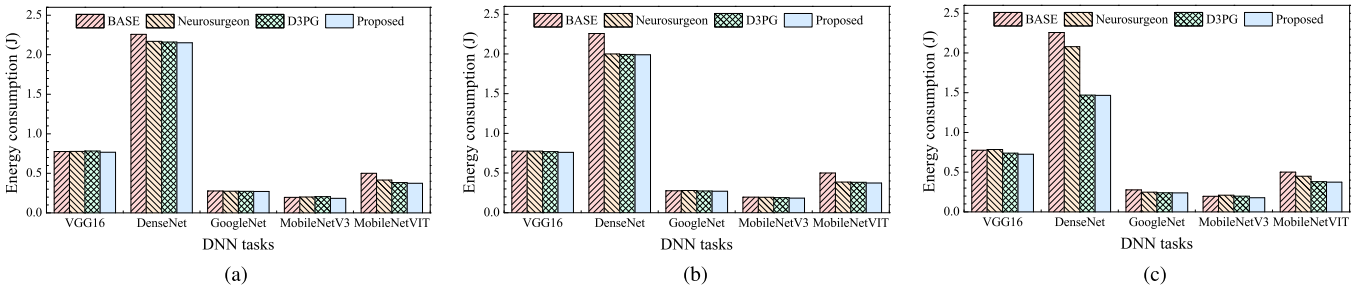


Fig. 10. Compare the energy consumption of executing different DNN tasks on Xavier NX using four methods under different time threshold ratios (θ). (a) $\theta = 0.1$. (b) $\theta = 0.2$. (c) $\theta = 0.3$.

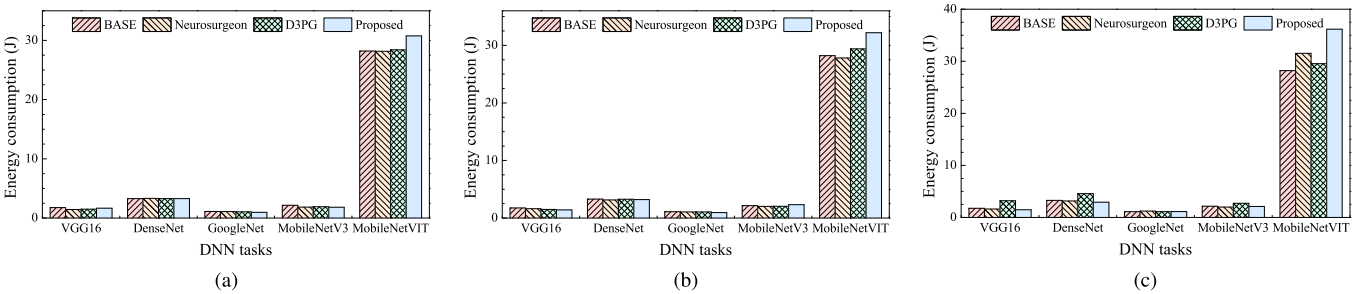


Fig. 11. Compare the energy consumption of executing different DNNs on Atlas 200I DK A2 using four methods under different time threshold ratios (θ). (a) $\theta = 0.1$. (b) $\theta = 0.2$. (c) $\theta = 0.3$.

reduce energy consumption by up to 35.77%, 33.41%, and 12.27% on average, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. It can be seen from the experiment results that as the time threshold ratio θ increases, the improvement in the performance of the proposed method becomes increasingly more significant. This is because when θ increases, the time constraint becomes more relaxed, and the proposed method has more space to dynamically adjust the multiple partitioning points.

Fig. 10 shows the energy consumption of executing different DNN tasks on Xavier NX using the BASE, Neurosurgeon, D3PG, and the proposed methods under different time threshold ratios. It can be seen from the figure that the proposed IL-based DNN partition method also achieves lower energy consumption on this device. When θ is set to 0.1, the proposed method can reduce energy consumption by up to 7.81%, 4.27%, and 2.97% on average, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. When θ is set to 0.2, the proposed method can reduce energy consumption by up to 9.42%, 2.84%, and 1.6% on average, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. When θ is set to 0.3, the proposed method can reduce energy consumption by up to 17.91%, 14.42%, and 2.81% on average, compared with the BASE, Neurosurgeon, and D3PG methods, respectively. From Figs. 8 to 10, we can see that the proposed IL-based DNN partitioning method achieves lower energy consumption than the BASE, Neurosurgeon, and D3PG methods on three devices.

Although the proposed IL-based DNN partitioning method is designed for GPU–CPU heterogeneous devices, it is necessary to discuss its scalability on other types of devices. We conduct experiments on one Huawei NPU platform (Atlas 200I DK A2). The energy consumption of executing different DNN tasks on the Huawei NPU platform using the BASE, Neurosurgeon, D3PG, and the proposed methods under different time threshold ratios is shown in Fig. 11. It can be seen from the figure that unlike the experimental results on the NVIDIA GPU–CPU heterogeneous devices, although the proposed IL-based DNN partitioning method achieves lower energy consumption than the other three benchmarking methods in some cases, it does not always achieve the lowest energy consumption. For example, when θ is set to 0.1, the proposed method outperforms the BASE, Neurosurgeon, and D3PG methods when partitioning GoogleNet and MobileNetV3, but the energy consumption of the proposed method is higher when partitioning MobileNetVIT. This is because our method is originally designed with GPU and CPU architectures, leveraging their specific characteristics. NPUs differ fundamentally in several aspects (e.g., computational architecture, execution model, memory architecture, and data movement) that significantly impact the performance of our approach. The experimental results demonstrate that achieving optimal performance across different accelerator architectures requires specialized implementations rather than a one-size-fits-all approach.

D. Experimental Results on Memory Usage

In addition to energy consumption, memory usage is also an important factor that affects the performance of DNN tasks on resource-constrained edge devices. To evaluate the memory usage of the proposed IL-based DNN partitioning method, we compare the memory usage of executing different DNN tasks on AGX Orin, AGX Xavier, Xavier NX, and Huawei

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I

MAIN FEATURES OF THE SELECTED DNN TASKS

DNN task	Train/Test	Number of layers
AlexNet	Train	20
U-Net	Train	69
ResNet18	Train	75
ResNet34	Train	139
ResNet101	Train	467
ResNet152	Train	705
VGG13	Train	29
VGG19	Train	53
SqueezeNet	Train	57
MobileNetV2	Train	138
VGG16	Test	44
DenseNet	Test	12
GoogleNet	Test	139
MobileNetV3	Test	138

TABLE II

MEMORY USAGE OF EXECUTING DIFFERENT DNN TASKS ON AGX ORIN USING FOUR METHODS (UNIT: MB)

Memory usage (MB)	BASE	Neurosurgeon	D3PG	Proposed
DenseNet	6.3285	6.4298	6.5832	6.3253
VGG16	6.4363	6.2542	6.3285	6.2534
GoogleNet	6.2371	6.58	6.7694	6.3155
MobileNetV3	6.9392	7.6413	8.376	7.1188
MobileNetVIT	9.9173	10.0153	10.0956	10.0022

NPU platforms using the BASE, Neurosurgeon, D3PG, and the proposed methods. The memory usage of executing different DNN tasks when $\theta = 0.1$ on AGX Orin using the four methods is shown in Table II. It can be seen from the table that compared with the Neurosurgeon and D3PG methods, the proposed method can slightly reduce the memory usage. However, compared with the BASE method, the memory usage of the proposed method is slightly higher in some cases. This is because the BASE method directly runs the unpartitioned DNN tasks on the GPU–CPU heterogeneous device, which does not require additional memory for partitioning and data transmission. Compared with the Neurosurgeon and D3PG methods, the proposed method can reduce the memory usage 2.53% and 5.55% on average, respectively. This is because compared with the Neurosurgeon method, the proposed method can dynamically adjust the DNN execution path to improve both energy consumption and memory usage. Compared with the D3PG method, the proposed method does not require a large memory for executing the deep RL models, and therefore can reduce the memory usage.

Tables III and IV show the memory usage of executing different DNN tasks on AGX Xavier and Xavier NX using the four methods when $\theta = 0.1$. It can be seen from the tables that similar to the experimental results on AGX Orin, the proposed method can slightly reduce the memory usage in most cases. For example, on AGX Xavier, compared with the BASE, Neurosurgeon, and D3PG methods, the proposed method can reduce the memory usage by up to 19.96%, 1.14%, and 0.41%, respectively. On Xavier NX, compared with the BASE, Neurosurgeon, and D3PG methods, the proposed method can reduce the memory usage by up to 27.83%, 5.55%, and 5.48%, respectively.

TABLE III

MEMORY USAGE OF EXECUTING DIFFERENT DNN TASKS ON AGX XAVIER USING FOUR METHODS (UNIT: MB)

Memory usage (MB)	BASE	Neurosurgeon	D3PG	Proposed
DenseNet	4.3758	3.9695	3.9663	3.9502
VGG16	4.9756	3.9502	3.9211	3.9824
GoogleNet	3.9276	3.9727	3.9437	3.9276
MobileNetV3	3.9792	4.1049	4.1017	4.1017
MobileNetVIT	4.3274	4.1694	4.1856	4.1856

TABLE IV

MEMORY USAGE OF EXECUTING DIFFERENT DNN TASKS ON XAVIER NX USING FOUR METHODS (UNIT: MB)

Memory usage (MB)	BASE	Neurosurgeon	D3PG	Proposed
DenseNet	17.4617	19.7228	19.7082	18.6287
VGG16	28.7673	20.6126	20.4084	20.7585
GoogleNet	15.8278	16.5864	15.2881	16.3238
MobileNetV3	16.1196	15.9153	15.9008	16.1779
MobileNetVIT	16.28	17.3741	17.5929	17.4033

TABLE V

MEMORY USAGE (NPU-DEDICATED MEMORY/DDR RAM) OF EXECUTING DIFFERENT DNN TASKS ON ATLAS 200I DK A2 USING FOUR METHODS (UNIT: MB)

Memory usage (MB)	BASE	Neurosurgeon	D3PG	Proposed
DenseNet	4532/4698	4518/4722	4515/4725	4535/4745
VGG16	4490/4654	4512/4703	4503/4574	4522/4664
GoogleNet	4485/4672	4515/4621	4509/4625	4526/4621
MobileNetV3	4489/4650	4524/4674	4508/4638	4531/4670
MobileNetVIT	4514/4719	4545/5184	4529/4716	4553/5180

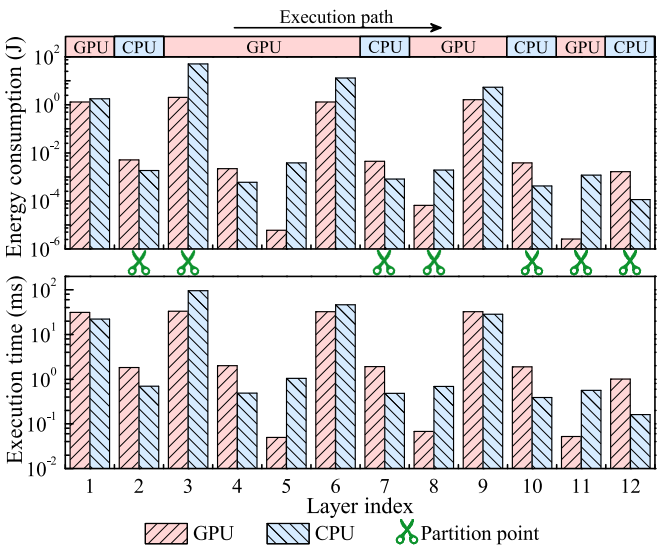


Fig. 12. Detailed partition process of the DenseNet on AGX Xavier when $\theta = 0.2$.

Table V shows the memory usage (NPU-dedicated memory/DDR RAM) of executing different DNN tasks on the Huawei NPU platform using the four methods when $\theta = 0.1$. Note that different from the NVIDIA GPU–CPU heterogeneous devices, the Huawei NPU platform is equipped with

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

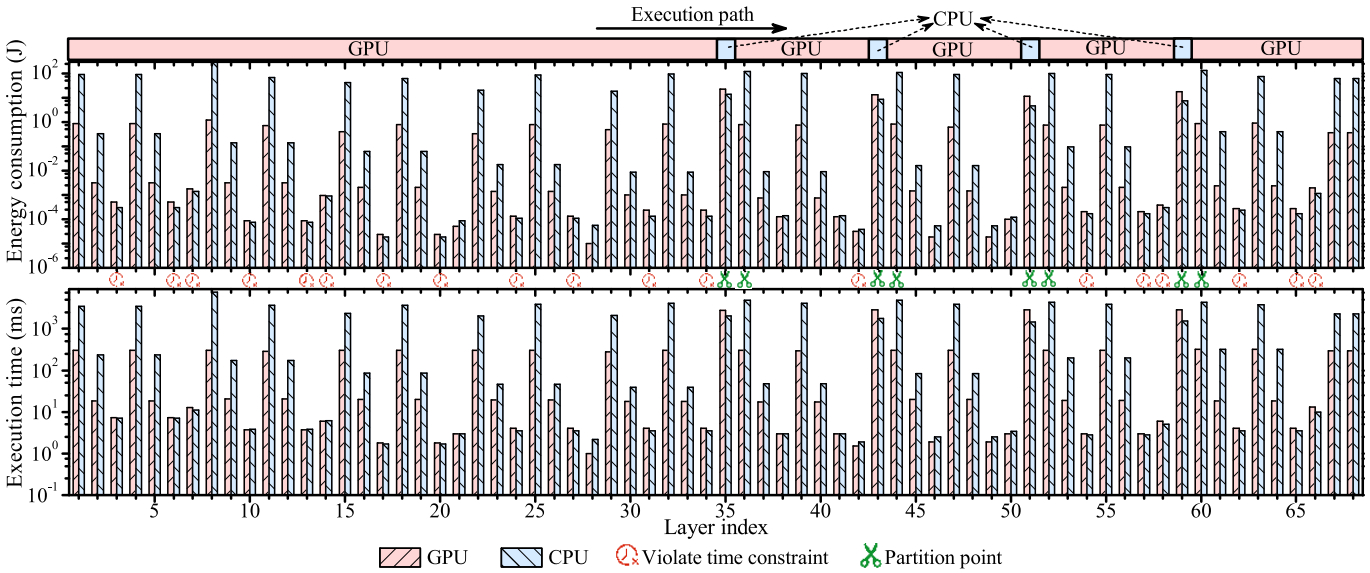


Fig. 13. Detailed partition process of the U-Net on AGX Xavier when $\theta = 0.2$.

an 8-GB dedicated NPU memory and an 8-GB DDR RAM. The memory usage of each DNN task is shown in the form of NPU-dedicated memory/DDR RAM in Table V. It can be seen from the table that the memory usage of the proposed method is slightly higher than that of the other three methods in most cases. This is because the proposed method is originally designed with GPU and CPU architectures, leveraging their specific characteristics. NPUs differ fundamentally in several aspects (e.g., computational architecture, execution model, memory architecture, and data movement) that significantly impact the performance of our approach. Our experimental results demonstrate that achieving optimal performance across different accelerator architectures requires specialized implementations rather than a one-size-fits-all approach.

E. Case Study: Time Constraints' Verification

In this section, we conduct a case study to demonstrate the partitioning process of the proposed IL-based DNN partitioning method and verify the time constraints. Fig. 12 shows the energy consumption (upper part) and the execution time (lower part) of each layer in U-Net executing on AGX Xavier when θ is set to 0.2. The X-axis represents the layer number, the yellow background represents the partition point obtained from the proposed IL-based DNN partitioning method, and the execution path is shown at the top of the figure. The energy consumption and execution time are obtained from the proposed *Profiler*. It can be seen from the figure that the number of partition points of the DenseNet task is seven, and the partition points are located at the 2nd, 3rd, 7th, 8th, 10th, 11th, and 12th layers. And the corresponding execution path is GPU (1st layer) → CPU (2nd layer) → GPU (3rd–6th layers) → CPU (7th layer) → GPU (8th–9th layers) → CPU (10th layer) → GPU (11th layer) → CPU (12th layer). The transmission data sizes between layers are small (all less than 1 MB), the transmission latency is negligible, and the energy consumption of the transmission is also negligible. From the figure, we can see that the GPU energy consumption of the 1st layer is larger than that of the CPU. The proposed algorithm, therefore, selects to execute the 1st layer on the GPU. The CPU energy consumption of the 2nd layer is larger than that of the GPU, so the proposed algorithm selects to execute the 2nd layer on the CPU. And the following partition points are selected according to the same principle. Note that in this case, there is no situation where the time constraint is violated.

Due to the limited computation resources of the GPU–CPU heterogeneous device, there may be a situation where the time constraints are violated. In this way, the proposed IL-based DNN partitioning method can adjust the partitioning strategy to meet the time constraints. We take the U-Net task as an example to demonstrate this situation. Fig. 13 shows the energy consumption (upper part) and the execution time (lower part) of each layer in U-Net executing on AGX Xavier when θ is set to 0.2. Similarly, the X-axis represents the layer number, the yellow background represents the partition point obtained from the proposed IL-based DNN partitioning method, and the execution path is shown at the top of the figure. Different from the case of DenseNet, the number of layers in U-net is 68, which is much larger than that of DenseNet. In this case, there are some situations where the time constraints are violated, as shown by the orange-shadowed background in Fig. 13. For example, it can be seen from the figure that the energy consumption of the 3rd layer in the GPU is larger than that of the CPU. Without considering time constraints, the 3rd layer should be executed on the CPU, while the 2nd layer is executed on the GPU. This implies that the 3rd layer would naturally serve as a partitioning point. However, due to the imposed time constraints, placing a partition at this point would result in a violation of the total latency requirement. Consequently, the proposed method does not introduce a partition at this layer. The number of partition points of the U-Net task is eight.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.

VII. Conclusions and Future Works

In this article, we propose an IL-based framework for dynamic DNN partitioning across GPU–CPU heterogeneous edge devices, aiming to minimize energy consumption while adhering to strict latency constraints. The proposed IL-based method consists of five components: 1) a *Profiler* for accurately predicting layerwise execution time and energy consumption; 2) a Dijkstra-based *Oracle Generator* for efficiently exploring optimal execution path; 3) a *Trainer* for training the partitioning model; 4) a *Predictor* for predicting the optimal execution path; and 5) a *Data Aggregator* for dynamically supplementing training data. Extensive experiments on NVIDIA Jetson platforms (AGX Orin, Xavier, and NX) demonstrate the effectiveness of the proposed IL-based partitioning scheme. Compared with three benchmarking methods, our method can reduce by up to 32.22% energy consumption. To discuss the method's scalability, we conduct the experiments on one Huawei NPU platform (Atlas 200I DK A2). We also conduct a case study to demonstrate the partitioning process and verify the time constraints. Note that the proposed IL-based DNN partitioning method can be easily extended by other designed *Oracle Generator*, which determines the quality of the training dataset and finally impacts the prediction performance.

In the future, we will further explore the scalability of the proposed method on other types of devices (such as FPGAs, NPUs, AMD, and Intel platforms) according to the characteristics of these devices. In addition, considering that factors such as battery status or network fluctuations can significantly impact the performance of DNN partitioning strategies, we identify the development of an online adaptive partitioning framework as a critical direction for our future work.

References

[1] J. Zhou et al., "Multiscale network traffic prediction method based on deep echo-state network for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21862–21874, Nov. 2022.

[2] Y. Qu, S. Huang, X. Chen, X. Wang, and Y. Yao, "Detection of backdoor attacks using targeted universal adversarial perturbations for deep neural networks," *J. Syst. Softw.*, vol. 207, Jan. 2024, Art. no. 111859.

[3] X. Xu, K. Yan, S. Han, B. Wang, X. Tao, and P. Zhang, "Learning-based edge-device collaborative DNN inference in IoVT networks," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7989–8004, Mar. 2024.

[4] H. Li, J. K. Ng, and T. Abdelzaher, "Enabling real-time AI inference on mobile devices via GPU-CPU collaborative execution," in *Proc. IEEE 28th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2022, pp. 195–204.

[5] T. Wang, K. Cao, J. Zhou, G. Zhang, and X. Wang, "Power-efficient layer mapping for CNNs on integrated CPU and GPU platforms: A case study," in *Proc. 26th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2021, pp. 627–632.

[6] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, "A survey of imitation learning: Algorithms, recent developments, and challenges," *IEEE Trans. Cybern.*, vol. 54, no. 12, pp. 7173–7186, Dec. 2024.

[7] J. Luo, P. Dong, Y. Zhai, Y. Ma, and S. Levine, "RLIF: Interactive imitation learning as reinforcement learning," 2023, *arXiv:2311.12996*.

[8] A. K. Kakolyris, M. Katsaragakis, D. Masouros, and D. Soudris, "RoaD-RuNNer: Collaborative DNN partitioning and offloading on heterogeneous edge systems," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Apr. 2023, pp. 1–6.

[9] J. Mao, X. Chen, K. Nixon, C. Krieger, and Y. Chen, "MoDNN: Local distributed mobile computing system for deep neural network," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, May 2017, pp. 1396–1401.

[10] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proc. 22nd Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2017, vol. 45, no. 1, pp. 615–629.

[11] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," 2021, *arXiv:2110.02178*.

[12] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.

[13] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 328–339.

[14] W. Zhang, N. Wang, L. Li, and T. Wei, "Joint compressing and partitioning of CNNs for fast edge-cloud collaborative intelligence for IoT," *J. Syst. Archit.*, vol. 125, Apr. 2022, Art. no. 102461.

[15] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2019, pp. 392–405.

[16] J. Zhang, W. Xin, D. Lv, J. Wang, G. Cai, and F. Dong, "Multi-exit DNN inference acceleration for intelligent terminal with heterogeneous processors," *Sustain. Comput., Informat. Syst.*, vol. 40, Dec. 2023, Art. no. 100906.

[17] C. Yu, T. Wang, Z. Shao, L. Zhu, X. Zhou, and S. Jiang, "TwinPilots: A new computing paradigm for GPU-CPU parallel LLM inference," in *Proc. 17th ACM Int. Syst. Storage Conf. ZZZ*, Sep. 2024, pp. 91–103.

[18] Y. Jiang, Y. Zhu, C. Lan, B. Yi, Y. Cui, and C. Guo, "A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters," in *Proc. 14th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2020, pp. 463–479.

[19] R. Kaur and F. Mohammadi, "Power estimation and comparison of heterogenous CPU-GPU processors," in *Proc. IEEE 25th Electron. Packag. Technol. Conf. (EPTC)*, Dec. 2023, pp. 948–951.

[20] Y. Wu, C. Wu, G. Yuan, Y. Li, W. Guo, and J. Rao, "DACO: Pursuing ultra-low power consumption via DNN-adaptive CPU-GPU co-optimization on mobile devices," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, Mar. 2024, pp. 1–2.

[21] M. Andrychowicz et al., "Learning to learn by gradient descent by gradient descent," in *Proc. Neural Inf. Process. Syst.*, Mar. 2016, pp. 3981–3989.

[22] (2021). *NVDIA Platforms*. [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/

[23] W. Jiang, Z. Song, J. Zhan, Z. He, X. Wen, and K. Jiang, "Optimized co-scheduling of mixed-precision neural network accelerator for real-time multitasking applications," *J. Syst. Archit.*, vol. 110, Nov. 2020, Art. no. 101775.

[24] Nvidia Corp. *NVlDlA Jetson: Embedded Systems for Accelerated Computing*. Accessed: Oct. 25, 2025. [Online]. Available: https://www.nvidia.com/en-sg/autonomous-machines/embedded-systems/

[25] Huawei Technologies Co. *Atlas 200I DK A2*. Accessed: Oct. 25, 2025. [Online]. Available: https://e.huawei.com/cn/products/computing/ascend/atlas-200

[26] Wikipedia. (2025). *Mean Squared Error*. [Online]. Available: https://en.wikipedia.org/wiki/Mean_squared_error

[27] L. Ale, S. A. King, N. Zhang, A. R. Sattar, and J. Skandaraniyam, "D3PG: Dirichlet DDPG for task partitioning and offloading with constrained hybrid action space in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 19260–19272, Oct. 2022.



Liying Li received the Ph.D. degree from the Department of Computer Science and Technology, East China Normal University, Shanghai, China, in 2022.

She is currently an Associate Professor with Nanjing University of Science and Technology, Nanjing, China. Her current research interests include areas of cyber-physical systems, IoT resource management, data mining, and distributed artificial intelligence.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.



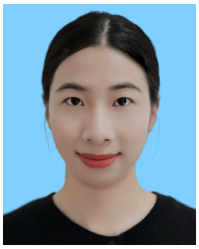
Tao Liu received the B.S. degree in computer science from Nanjing Forestry University, Nanjing, China, in 2024. He is currently pursuing the M.S. degree with Nanjing University of Science and Technology, Nanjing.

His research interests include edge computing and artificial intelligence of things.



Mingzhou Zhao received the M.S. degree from the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China, in 2024.

His research interests include embedded systems.



Peijin Cong is currently an Associate Professor with Nanjing University of Science and Technology, Nanjing, China. Her research interests are in the area of cloud computing, service computing, and IoT, where she has published 30 refereed papers, more than half of which appeared in IEEE/ACM TRANSACTION papers.



Zebin Wu (Senior Member, IEEE) was a Visiting Scholar with the GIPSA-Laboratory, Grenoble INP, Université Grenoble Alpes, Grenoble, France, from August to September 2018. He was a Visiting Scholar with the Department of Mathematics, University of California at Los Angeles, Los Angeles, CA, USA, from August to September 2016 and from July to August 2017. He was a Visiting Scholar with the Hyperspectral Computing Laboratory, Department of Computer Technology and Communications, Escuela Politécnica, University of Extremadura, Cáceres, Spain, from June 2014 to June 2015. He is currently a Professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China. His research interests include edge computing, parallel computing, hyperspectral image processing, and big data processing, where he has published more than 200 refereed papers.



Junlong Zhou (Member, IEEE) received the Ph.D. degree in computer science from East China Normal University, Shanghai, China, in 2017.

He was a Visiting Scholar with the University of Notre Dame, Notre Dame, IN, USA, from 2014 to 2015. He is currently a Professor at Nanjing University of Science and Technology, Nanjing, China. His research interests include edge computing, cloud computing, and embedded systems, where he has published 120 refereed papers, including over 40 in premier IEEE/ACM TRANSACTIONS.

Dr. Zhou received the Best Paper Awards from IEEE iThings 2020, IEEE CPSCom 2022, and IEEE ICITES 2024. He has been an Associate Editor of *Journal of Sustainable Computing: Informatics and Systems*, *Journal of Circuits, Systems, and Computers*, and *IET Cyber-Physical Systems: Theory and Applications*, a Subject Area Editor of *Journal of Systems Architecture: Embedded Software Design*, and a Guest Editor of eight journals such as *ACM Transactions on Cyber-Physical Systems*.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 10,2026 at 04:05:56 UTC from IEEE Xplore. Restrictions apply.